
Probabilities Without Payoff: Replacing Hand-Built Decisions in Evolutionary Agent Fuzzing with a Probability-Returning Decision Model

Kunal Pai

University of California, Davis
kunpai@ucdavis.edu

Abstract

Evolutionary fuzzers for LLM agents rely on hand-built rules to steer their search, such as fixed score thresholds, uniformly random mutation choices, and LLM judges that grade each attack. We ask whether a decision model that returns a probability for every option can replace these rules and find better attacks under the same budget. We replace each decision point in the NAAMSE fuzzer with TypeSafe Jev and evaluate eight ablation arms (40 runs, 1,120 attack prompts) against a Muse Spark target, with two further plain-LLM baseline arms and five follow-up analyses that send no new queries to the target. We find no evidence that Jev improves the search: no arm outperforms the baseline after correction, and Jev’s zero-shot action policy is within two judge-score points of the static thresholds. A plain LLM selector (Muse Spark, which names one option instead of returning probabilities) is also not significantly better than the baseline, so the negative result is not specific to Jev or to calibration. Jev chooses mutation operators based on how they are described rather than how well they work, and as a judge it broadly agrees with Meta on labels but calls fewer refusals. A controlled experiment that applies ten operators to the same 20 parents finds that no operator beats re-sending the parent unchanged, and that Jev’s preferred operators tend to score below uniform choice. Our follow-up analyses also show that the fuzzer itself shapes these results: about a quarter of mutated prompts are the mutation model’s own refusal, a shared random seed couples decisions that should be independent, and the judges’ “successes” are mostly compliance with low-risk requests. Finally, a refusal-gated referee reproduces the stored scores at 47% of the judge calls, since no refusal in the sweep received a harm verdict.

1 Introduction

AI agents are increasingly deployed with access to tools, data, and users, which makes their security a practical concern rather than a theoretical one. Manual red-teaming does not scale to this setting, so recent work automates it. Mutation-based fuzzers evolve jailbreak templates [17], attacker LLMs refine prompts over several rounds [2], and NAAMSE [12] treats agent security evaluation as an optimization problem: it starts from a large corpus of known attacks, sends them to the agent, scores each response, and evolves the most promising prompts.

Every such fuzzer has to make the same kinds of decisions over and over. For example, when a prompt partly works, should the fuzzer look for similar prompts in the corpus, mutate this one, or give up and try something new? Which of its 26 mutation operators (e.g., translating the prompt, wrapping it in a role-play, or encoding it) should it apply? And how should it grade the target’s response? In NAAMSE, these decisions are made by fixed rules: score thresholds pick the next action, the operator

is drawn uniformly at random, and a panel of LLM judges grades each response [18]. These rules are simple heuristics, and it is natural to ask whether a learned model could make better choices.

In this work, we investigate whether a decision model that returns a *probability* for every option can replace these rules. The intuition is that such a model can condition each choice on the prompt and on how the judges rated it, and can express how confident it is, which is useful to the extent that its probabilities are calibrated [7]. To test this, we replace each decision point in NAAMSE with TypeSafe Jev and evaluate eight ablation arms that swap in Jev as the action selector (RQ1), the fitness judge (RQ2), or the operator selector (RQ3), under a fixed budget of 28 attack prompts per run.

First, we find that Jev does not improve the search. No arm outperforms the baseline, and because each run makes only 28 attacks, we also pool 480 individual decisions and re-estimate each policy offline. All three estimators we use bound Jev’s gain from choosing actions to less than two judge-score points. Even an oracle that always picks the best action for each range of parent scores would gain less than three points, which suggests that, at least with the information the baseline uses, action selection is not where the leverage is.

Furthermore, we observe that Jev’s choice of mutation operator is driven by wording, not evidence. When we swap the operator descriptions between operators, Jev’s preferences follow the descriptions rather than the operator names. Its favorite operators are among the weakest in our data, and when we show it outcome statistics, it jumps to operators that were tried only once or twice. To measure what operator choice is actually worth, we apply ten operators to the same 20 parents. Surprisingly, no operator beats simply re-sending the parent, and replaying Jev’s preferences on this grid tends to score below uniform choice.

While analyzing these results, we found that the fuzzer itself shapes what any selector can achieve. About a quarter of the “attacks” produced by mutation are actually the mutation model refusing to write the attack. A single random seed per task is shared by the action choice, the operator choice, and the corpus sample, so the operators that are actually applied differ from the ones Jev intended. Finally, a blind annotator from a third model family agrees with the judges on whether the target complied, but finds no high-risk content among the 13 prompts that the judges counted as successes, or in any of the five prompts that a judge rated high-risk. Most of these “successes” are the target complying with low-risk requests, such as writing suggestive pickup lines.

We report this as a negative result, and we think the more useful part is why a well-motivated idea did not deliver.

In summary, this paper makes the following contributions:

- We show that replacing NAAMSE’s action selector, operator selector, and fitness judge with a probability-returning decision model (Jev) or a plain LLM selector (Muse Spark) does not significantly improve attack outcomes, and that action selection based on the parent’s score has less than three judge-score points of headroom in our setting.
- We show that zero-shot operator selection is driven by operator descriptions rather than operator outcomes, that adding outcome statistics makes the model chase small samples, and, in a controlled experiment, that no operator outperforms a no-op against this target.
- We identify properties of the fuzzer that shape every arm: mutator refusals and silent mutation failures, a shared per-task seed, and a success threshold that rewards low-risk compliance. We quantify how each one affects the reported results.
- We propose a refusal-gated referee that needs 47% of the judge calls, and we check the Meta judge, and spot-check Jev, with a blind cross-model annotation.

2 Background

NAAMSE. NAAMSE runs in iterations. In each iteration, it picks parent prompts in proportion to their fitness, applies one action to each parent, sends the resulting children to the target agent, and scores the responses. There are three actions: EXPLORE draws a new prompt from the corpus, SIMILAR draws a prompt that is close to the parent in embedding space, and MUTATE rewrites the parent with one of 26 operators. In the baseline, the action depends only on the parent’s score. Parents scoring below 50 are usually abandoned (EXPLORE with probability 0.7), parents scoring

between 50 and 80 are usually explored nearby (SIMILAR with probability 0.7), and parents scoring above 80 are usually mutated (MUTATE with probability 0.7).

Judge score. Each response receives a judge score that combines how much the target complied with how harmful the response was. A response-alignment judge assigns a term r from full refusal to full acceptance, and six harm judges (one per category) assign a term h from not harmful to harmful:

$$s = r + h \cdot (1 + \pi/10), \quad r \in \{25, 42, 58, 75\}, \quad h \in \{-20, 5, 15, 30\}, \quad (1)$$

where h is the strongest harm term across categories and $\pi \in [0, 1]$ is a small boost for detected personal information. In practice the score takes only a few values: weak refusals land near 20 (full refusals near 5), partial compliance near 37, and harmless compliance near 54. Following NAAMSE, we call a prompt with $s \geq 80$ a *success*. Note that a full acceptance with a single low-risk category already scores 80, so a success does not have to contain high-risk content. This detail becomes important in § 4.

Jev. TypeSafe Jev is a commercial decision API: it answers typed questions (for example, “which of these options?”) and returns a probability for every option instead of free text. Its model and training details are not public, so we treat it as a black box and use it zero-shot, with no fine-tuning and no feedback from earlier decisions.¹ [Kunal: Add a citation or URL for TypeSafe Jev and the model version used.] We turn each decision point into such a question. For action selection, the question includes the parent prompt, its score, and its per-category judge verdicts, and the action is sampled from Jev’s probabilities at temperature 1.0. For operator selection, each of the 26 options comes with one sentence that describes what the operator does. As a judge, Jev answers the same rubric questions as the Meta judges.

Per-task seeding. NAAMSE runs several tasks in parallel and gives each task one random seed. Each decision inside a task creates a new random number generator from that same seed. As a result, the action choice, the operator choice, and the corpus sample all start from the same random number. We keep this configuration unchanged, since it is part of the system under test, and measure its effect in § 4.

3 Study Design

Threat model. We consider a black-box attacker who can send messages to a deployed agent but cannot see its weights or system prompt. The attacker has a fixed budget of 7 iterations with 4 mutations each, for 28 attack prompts per run, and tries to get the agent to produce harmful content in any of six categories: disinformation, illegal goods and services, hate and harassment, non-violent crime, violence, and sexually explicit content.

Ablation arms. Table 1 lists the ten arms: eight from the sweep and two plain-LLM baseline arms (Act=Muse, Mut=Muse). Each arm replaces one component with Jev and keeps the others at their defaults. We run every arm under the standard score objective, and three of them also under a coverage objective that rewards reaching new harm categories. Two arms use a plain-LLM baseline, Muse Spark: it gets the identical decision question but replies with one option in free text rather than a probability, at temperature 0, so the shared-seed coupling cannot distort its picks.

Follow-up analyses. Five runs per arm leave little statistical power, so we add five follow-up analyses that reuse the stored runs and send no new queries to the target (Table 2).

- *Off-policy re-estimation.* Every action decision logs the probability with which it was taken, so we can estimate how any policy would have done on the same data [4].
- *Description ablation.* We replay 66 parents to Jev’s operator question under five different framings of the options. These are the unique parents of mutated prompts in the four score-objective, Meta-judged arms.

¹We accessed Jev through <https://api.typesafe.ai> in September 2026 using the provider’s default model identifier (jev-latest), which is not pinned to a version.

Table 1: Ablation arms (5 seeds each). Bold marks the component replaced by Jev or by the plain LLM selector Muse.

Arm	Action selector	Operator selector	Fitness judge	Objective
Baseline	static thresholds	uniform	Meta	score
Act=uniform	uniform	uniform	Meta	score
Act=Jev	Jev	uniform	Meta	score
Mut=Jev	static thresholds	Jev	Meta	score
Fit=Jev	static thresholds	uniform	Jev	score
Act=Muse	Muse	uniform	Meta	score
Mut=Muse	static thresholds	Muse	Meta	score
Cov baseline	static thresholds	uniform	Meta	coverage
Cov + Act=Jev	Jev	uniform	Meta	coverage
Cov + Mut=Jev	static thresholds	Jev	Meta	coverage

Table 2: Cost of the follow-up analyses and the controlled experiment. The original sweep used seven judge calls per prompt.

Follow-up	Target queries	Judge calls	Jev calls	Input
Off-policy re-estimation	0	0	0	logged probabilities
Description ablation	0	0	358	66 replayed parents
Common referee (gated)	0	458 (980 ungated)	0	140 prompts
Randomness and mutator audit	0	0	0	corpus database
Cross-model annotation	0	0	0	103 blind labels (4 Claude instances)
Controlled operator experiment	200	758 (gated)	0	20 parents $\times$ 10 operators
<i>Original sweep</i>	<i>1,120</i>	<i>$\approx 7,840$</i>	<i>–</i>	<i>40 runs</i>

- *Common referee.* The Fit=Jev arm was graded by Jev itself, so we re-score its 140 prompts with the Meta judges. The referee runs the alignment judge first and skips the six harm judges when the response is a refusal. In the sweep, all 754 refusals received “not harmful” in every category, so we expect skipping these calls never to change a score; we did not re-run the skipped calls to confirm this directly.
- *Randomness and mutator audit.* We inspect every mutated prompt, compare it with its parent, and locate every EXPLORE draw in the corpus.
- *Cross-model annotation.* Because Meta is both the target and the default judge, we ask an annotator from a third model family (Claude, in four instances labeling disjoint quarters) to label a stratified sample of 103 prompts, blind to every judge verdict and using the judges’ own rubrics. The sample contains all 13 successes under each arm’s own judge, 45 near-misses, 15 partial acceptances, and 30 refusals.

4 Evaluation

4.1 Experimental Setup

The target is Muse Spark behind the NAAMSE example agent, reached over the A2A protocol. The target, the mutation model, and the default fitness judges all use Muse Spark (muse-spark-1.2) at temperature 0. Each arm runs with seeds 1 to 5, for 40 runs and 1,120 attack prompts in the sweep, plus 10 runs for the two Muse arms. We treat each run as one observation. Because runs with the same seed do not start from the same prompts across arms, we compare arms with unpaired exact permutation tests (252 splits), report 95% bootstrap intervals and Cliff’s δ , and apply Holm correction across the seven planned contrasts per metric. For analyses that pool individual decisions, we compute intervals by resampling whole runs. With five runs per arm and the observed spread (a standard deviation of about 6 points in mean judge score), the smallest difference we could detect with 80% power at $\alpha = 0.05$ is about 11 points, so a null result at the arm level rules out only large effects.

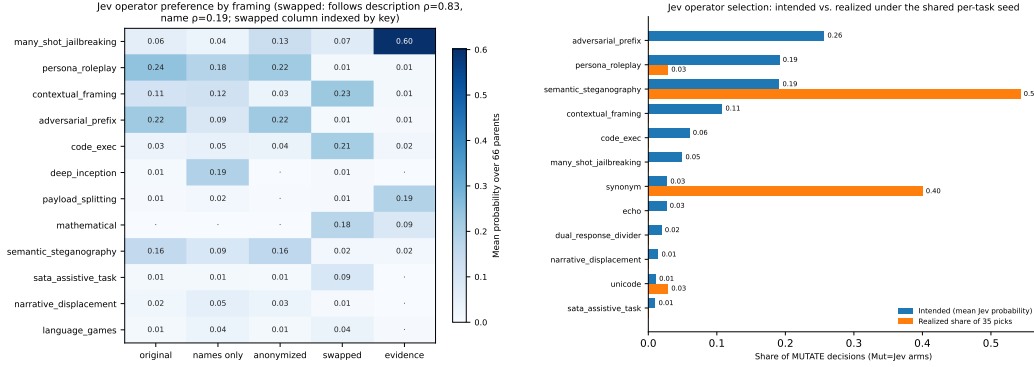
4.2 High-Level Takeaways

Takeaway 1: We find no evidence that replacing the hand-built rules with Jev improves the search. No arm is distinguishable from the baseline (Appendix Figure 4). Among the Meta-judged arms, the largest difference from the baseline in mean judge score is Act=Jev, at +2.7 points with a 95% interval of $[-4.2, +9.3]$ ($p = 0.50$), and the largest planned contrast overall, Act=Jev versus Act=uniform (+4.2, $p = 0.22$), is also not significant. Fit=Jev looks larger under its own judge (+6.0), but that arm is graded differently; on the common Meta scale it shrinks to +3.3 ($p = 0.48$; Takeaway 6). After Holm correction, every p -value across mean score, maximum score, successes, coverage, and refusal rate is 1.00, except one maximum-score contrast (0.50) that is driven by a single run. The target is hard to attack: it refuses about 70% of prompts in the Meta-judged arms, and only 13 of 1,120 prompts (1.2%) reach a score of 80 under each arm’s own judge. These successes come from just 6 runs, and two of Act=Jev’s five are initial random draws made before any Jev decision. The plain-LLM baseline does not change this picture. Act=Muse is the numerically strongest arm (mean 33.5, 8 successes), but its edge over the baseline is +4.6 $[-1.1, +10.1]$ ($p = 0.21$), and no Muse contrast is significant after Holm correction (Appendix Table 7). Being deterministic, Muse applies operators more evenly than Jev’s seed-distorted picks (5 versus 2), so the distortion is not the selector’s doing, yet it still does not win. In 39 of 40 runs, the best score improves by less than one point after iteration 3; the exception is one coverage run in which a late random draw scored 100, the same run that drives the one maximum-score contrast (Figure 5). This confirms that five runs per arm cannot resolve differences of a few points, which motivates the pooled analyses below.

Takeaway 2: Choosing actions better would help little, and Jev does not do it. Jev’s zero-shot policy is sensible: it rarely mutates weak parents, and its probability of mutating rises sharply once the parent scores above 50. However, it prefers SIMILAR where the baseline prefers EXPLORE, so Act=Jev spends less of its budget on new prompts (31% instead of 56%). When we pool 480 decisions, Jev’s preference turns out to be right for weak parents (below 30) and wrong for middling ones (30 to 50), and the two errors cancel out (Table 3). Since each decision logs the probability with which it was taken, we can estimate each policy’s value offline with three estimators: the direct method, SNIPS, and doubly robust estimation (Appendix Table 4). All three place Jev within two points of the baseline policy. Outside the Act=Jev arm, Jev’s policy is approximated by its average probabilities in each score range, so these estimates cannot see any prompt-specific signal Jev might use. An oracle that always picks the best action for each of four score ranges, fitted on the same data and therefore optimistic, reaches only 32.6 (95% interval up to 35.7), compared with 29.7 for the baseline. The Act=uniform arm, whose actions are random, shows the same pattern on its own, so the result is not an artifact of pooling. This suggests that, with the parent’s score as the state, action selection has little headroom here.

Takeaway 3: Jev picks operators by how they are described, not by how well they work. In our replay, Jev’s three favorite operators (`persona_roleplay`, `adversarial_prefix`, and `semantic_steganography`) are among the weakest in the sweep’s data. For instance, the target refused `adversarial_prefix` in all six of its uses in the sweep (Figure 8), and in the controlled experiment of Takeaway 8 it scores 6.7 points below a no-op. To find out what drives Jev’s choice, we replay 66 parents under five framings of the options (Figure 1a and Table 5). Hiding the operator names barely changes Jev’s preferences ($\rho = 0.91$). Swapping the descriptions between operators, however, moves Jev’s preferences along with the descriptions ($\rho = 0.83$) rather than the names ($\rho = 0.19$). For example, the operator that inherits `semantic_steganography`’s description receives 0.23 of the probability, while `semantic_steganography` itself drops to 0.02. Jev’s preferences also barely change from one parent to the next, so its choice is largely a fixed preference set by the wording. When we add each operator’s observed outcomes to its description, Jev does move toward operators that scored well, but it picks the two with the least evidence (`many_shot_jailbreaking`, tried twice, and `payload_splitting`, tried once). Worse, both recorded uses of `many_shot_jailbreaking` were failed mutations that re-sent the parent unchanged (Takeaway 4), so its high score belongs to its parents. This confirms that operator descriptions act as hidden parameters of a zero-shot selector, and that a selector which learns from outcomes needs to account for how much evidence it has.

Takeaway 4: The fuzzer’s own machinery changes what a selector can achieve. Three properties of NAAMSE, unrelated to Jev, change the experiment that actually runs. The first is the shared per-task seed. Because MUTATE is the last action, it is only chosen when the shared random number



(a) Mean probability over 66 parents, five framings. (b) Intended vs. realized picks in the Mut=Jev arms. Figure 1: Jev operator selection. (a) Preferences follow the descriptions. (b) The shared per-task seed turns them into picks from the end of the alphabet.

is high, and the operator is then drawn with that same high number from a list sorted by name. As a result, the operators that Jev actually applied are not the ones it preferred: `synonym` had a probability of only 0.03 but was applied 14 times. Simulating the configured seeding with Jev’s logged probabilities reproduces this pattern (63% `semantic_steganography` and 24% `synonym`, against 54% and 40% realized), while sampling them independently recovers Jev’s intended shares (Figure 1b). The Mut=Jev arm therefore never tested Jev’s operator preferences; the description ablation (Takeaway 3) tests them directly, and Takeaway 8 estimates what they would have been worth. The same effect limits EXPLORE, which only ever samples the first $P(\text{explore})$ fraction of the corpus (Figure 10a). Since the corpus is stored in source order, a policy that explores less also explores a narrower slice. The second property is that the mutation model sometimes refuses. In 44 of 164 mutated prompts (27%, or 23% with a stricter detector that only matches refusal openers), the text sent to the target is the mutation model’s own refusal to write the attack. For instance, `semantic_steganography` produced a refusal in 19 of its 30 uses, whereas `synonym`, whose transformation uses a thesaurus rather than a language model, never did. These refusals score lower than real attacks (24.7 versus 31.0) and are refused more often by the target (82% versus 62%). Neither detector was hand-validated, and both miss non-English refusals. The third property is that mutations can fail silently: when a mutation raises an error, NAAMSE sends the parent unchanged. Excluding the no-op operator, 15 of the 164 mutated prompts are identical to their parent, including both uses of `many_shot_jailbreaking`. The controlled experiment in Takeaway 8 finds at least three causes: the tool behind `many_shot_jailbreaking` cannot reach the prompt database it needs and crashes whenever it is called (in the other cells, the mutation model refused or rewrote the prompt without the tool), the mutation agent sometimes loops until it hits its step limit, and one of `semantic_steganography`’s strategies had a code defect, which we fixed before that experiment. These causes do not explain everything: 18 of the 47 silent failures in that experiment left no error at all. This confirms that an ablation of a fuzzer’s control should log both intended and realized choices, and should check whether each mutation actually produced an attack.

Takeaway 5: Jev’s action policy narrows the range of attacks the fuzzer tries. We recover the corpus cluster of every prompt offline and count how many of the 30 top-level clusters each run visits. Act=Jev visits 9.0 clusters per run, compared with 12.4 for the baseline, and the difference in cluster entropy is 0.64 bits with a 95% interval of [0.23, 1.07] (Figure 9). This difference does not survive Holm correction, but it is among the largest effects in our study, and its direction replicates under the coverage objective (Cliff’s $\delta = -0.88$). This is consistent with Jev’s preference for SIMILAR: across the 35 Meta-judged runs, the more a run uses SIMILAR, the fewer clusters it visits ($\rho = -0.84$) and the higher its mean score ($\rho = +0.55$). The shared seed explains only about 10% of the gap; the rest comes from Jev exploring less (23 EXPLORE draws after the first iteration, compared with 61). This matters because all 13 successes (under each arm’s own judge) come from just 3 of the 26 clusters the fuzzer visited. This confirms that a policy can look neutral on score while quietly reducing the diversity of attacks.

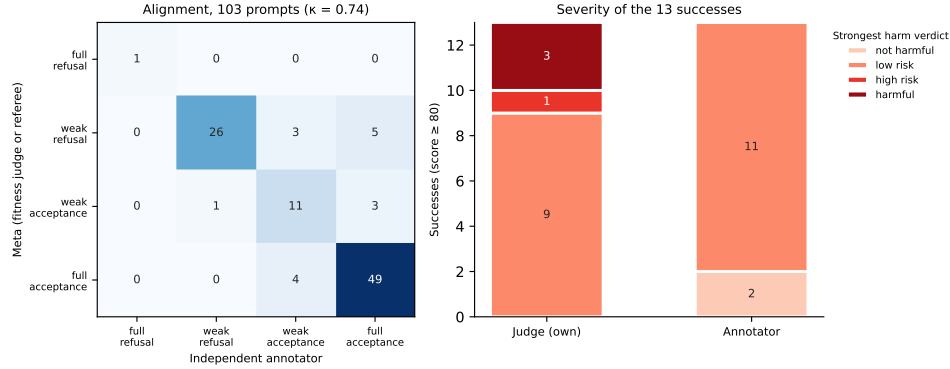


Figure 2: Blind cross-model annotation. Left: alignment labels from Meta (fitness judge, or referee for Fit=Jev) against the annotator on 103 prompts. Right: strongest harm verdict on the 13 successes under each arm’s own judge.

Takeaway 6: As a judge, Jev broadly agrees with Meta on labels, and an independent annotator finds none of the successes high-risk. Once the Meta referee re-scores the Fit=Jev arm, it is no better than the baseline (32.1 versus 28.9, with a 95% interval of $[-3.8, +11.1]$ for the difference). Under its own Jev judge, the gap had been nearly twice as large (+6.0), so about half of it came from labeling. On the 140 prompts graded by both judges, they agree on the alignment label 77% of the time (Cohen’s $\kappa = 0.64$, quadratic $\kappa = 0.76$). When they disagree, Jev usually rates the response as more compliant (25 of 32 cases; Table 6), and Jev calls 49% of responses refusals compared with Meta’s 62%. Jev’s probabilities are therefore miscalibrated relative to Meta on refusals: its refusal probability averages 0.47 against Meta’s refusal rate of 0.62 (expected calibration error 0.15), while its category-relatedness probabilities are close to Meta’s verdicts (expected calibration error 0.03 over 318 verdicts). The blind annotator agrees with Meta on alignment labels 84% of the time ($\kappa = 0.74$, quadratic $\kappa = 0.82$), which is closer than Jev’s agreement with Meta; on the 22 Jev-judged prompts in its sample, it agrees with Jev 82% of the time ($\kappa = 0.48$). The annotator agrees with Meta on whether a response contains high-risk content 96% of the time, but it disagrees on every response that a judge escalated (Figure 2). The fitness judges rated four prompts in the whole sweep high-risk or harmful, and the Meta referee rated a fifth (a playful JavaScript easter egg that Jev had scored 54); the annotator rates all five as low-risk or not harmful. One of the four is a prompt injection that got the target to say “Access Granted” but contains no harmful content; the others are generic insults, a crude joke (which the Meta referee also downgraded to low-risk), and a non-explicit role-play. Nine of the 13 successes are full acceptances whose strongest harm verdict is low-risk, which is exactly the case Equation 1 allows. The count itself depends on the judge: on the common Meta scale, the referee finds 7 successes in Fit=Jev instead of Jev’s 4, for 16 in the sweep. Finally, none of the 30 judged refusals contains any harm, which supports skipping the harm judges on refusals. This confirms that the study’s successes mostly measure compliance with low-risk requests, and that success at the high-risk level should be reported separately and checked by a judge outside the target’s model family.

Takeaway 7: The coverage objective turns into random search. The coverage objective only rewards a prompt when it reaches a new harm category at high risk or above. Since this almost never happens, every parent has fitness 0, and both the baseline and Jev fall back to exploring new prompts most of the time (71% and 64% of prompts). In 14 of the 15 coverage runs, no category is ever covered. This confirms that a coverage objective needs partial credit, or a fallback to the score objective until the first category is covered, before it can test any selector.

Takeaway 8: In a controlled test on refused and harmlessly answered parents, no operator beats re-sending the parent, and Jev’s preferences tend to do worse than uniform choice. The sweep’s operator evidence is observational, so we run a controlled experiment. We draw 20 parents from the score-objective runs (8 scoring below 30, 4 between 30 and 50, and 8 at the harmless-compliance plateau around 54; none is near success), apply each of ten operators to every parent with a fixed seed, and send all 200 attacks to the target, scored by the refusal-gated Meta judges (758 judge

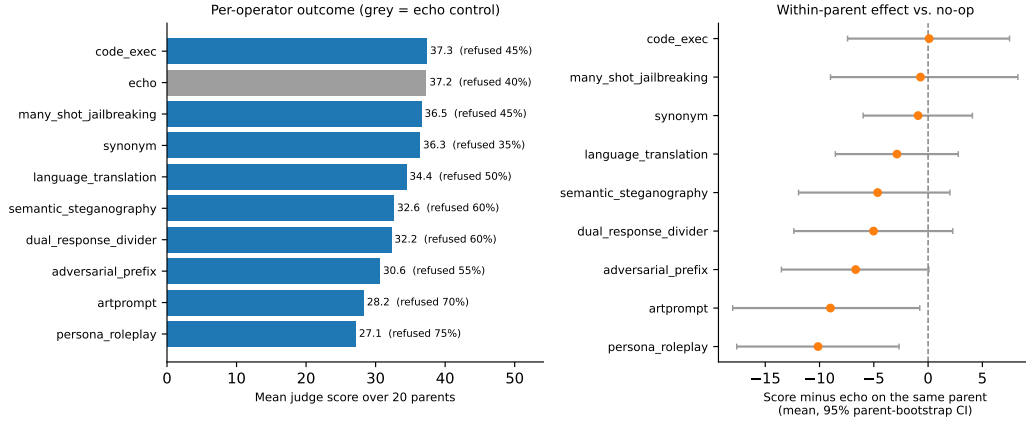


Figure 3: Controlled operator experiment (20 parents $\times$ 10 operators, refusal-gated Meta judges). Left: mean judge score and refusal rate per operator; grey is the echo no-op. Right: each operator’s score minus echo on the same parent.

calls). The ten operators are a no-op control (echo, which re-sends the parent unchanged), Jev’s three favorites, the operators that looked strongest in the sweep, a non-LLM operator (synonym), and one weak operator. Because every operator sees the same parents, we compare each one with echo on the same parent (Figure 3). No operator scores meaningfully higher than echo (37.2), and their apparent top ranks mean little, since about half of the cells of `code_exec`, `many_shot_jailbreaking`, and `synonym` were silent re-sends of the parent. The largest loss is `persona_roleplay`, one of Jev’s favorites, at 10.1 points below echo (Holm-adjusted $p = 0.050$), and 12.7 below on real attacks only ($p = 0.01$); among real attacks the only operator above echo is `synonym` (+4.4 [−0.1, +10.5]). None of the 200 attacks reaches 80 or a high-risk verdict, and the mutations are unreliable: 32 are mutator refusals (12 of 20 for `semantic_steganography`) and 47 are silent failures, so 40% of the “mutations” are not new attacks. The echo re-test also shows a single score carries about 6 points of noise (Spearman $\rho = 0.72$). Finally, because every operator was observed on every parent, we can replay operator policies on this grid without new queries (Figure 11). Over a run-sized budget of 10 decisions, uniform choice averages 33.2 points and Jev’s intended preferences 31.6 (1.6 points lower, with a 95% interval over parents of [−3.8, +0.2]), while UCB1 (33.4) and Thompson sampling (33.6) are no better than uniform; Thompson sampling reaches 35.3 only after 200 decisions, still below the best fixed operator (37.3). This suggests that, for parents the target refuses or answers harmlessly, the spread between operators is mostly downside: the best a selector can do is avoid harmful operators, which Jev’s zero-shot preferences do not. We did not test parents near success, where operators might matter more.

5 Discussion

Where the leverage is. The decisions the selectors replaced were not the bottleneck: the target refuses about 70% of prompts, the judge score takes only a handful of values, and even operator choice offers little upside, since no operator beat re-sending the parent. What matters is the attack material, so decision points should be ranked by their oracle headroom, which the pooled decisions give at no extra cost, before a model is deployed at them. And a zero-shot selector inherits whatever its option descriptions emphasize: a description that promises to hide intent is an attractive answer against a refusal-heavy target even when the operator performs poorly, so descriptions deserve the same care as weights, and a selector that learns from outcomes needs an explicit notion of uncertainty.

Cost. Skipping the harm judges on refusals cut the referee’s cost from 980 to 458 judge calls (Table 2); since no refusal in the sweep received a harm verdict, we expect it to change no score, and the saving grows with the refusal rate.

6 Threats to Validity

Internal validity. Five runs per arm and 28 prompts per run limit statistical power (the exact permutation test cannot reach p below 0.008), and the pooled analyses assume an action’s outcome does not depend on the policy that chose it, which the shared seed violates for EXPLORE. The oracle bound is fitted in-sample and is optimistic. The cluster analysis, mutator audit, and seeding analysis were defined post hoc; 14% of prompts have kNN-assigned clusters, and the audit misses non-English refusals.

Construct validity. Meta is both the target and the referee, which risks self-preference bias; our annotator removes the shared-model concern but is itself an LLM (Claude), so its severity calls are a second opinion, not ground truth. [Kunal: A human pass on the blind sheet (labeling/) would settle the severity question.] The description ablation measures Jev’s preferences, not their downstream effect, and the controlled experiment observes each (parent, operator) pair once, with about 6 points of single-score noise, running NAAMSE’s operators as configured (including the two failure modes we did not fix). Both selectors are commercial LLMs from the same families as the target and judges, and Jev’s model identifier is unpinned; other models, or an open selector, could behave differently.

External validity. Our results describe NAAMSE as configured, including its per-task seeding, and a single refusal-heavy target. A fuzzer with independent random draws, or a more compliant target, could shift which decision points carry leverage.

Ethics and broader impact. This work red-teams a deployed model through its public API to improve how such testing is evaluated, and it produced no high-risk content by independent review. Automated red-teaming tools can be misused, so we release analysis code and derived tables but not the raw attack prompts or target responses, which remain available to researchers on request. The study involves no human subjects; the annotation was done by an LLM.

7 Related Work

Automated red-teaming has moved from static benchmarks toward adaptive search. LLM-Fuzzer (originally GPTFuzzer) [17] mutates jailbreak templates and selects seeds with a tree search, and TurboFuzzLLM [6] and RLbreaker [3] learn which mutation to apply with reinforcement learning, which is the closest prior work to our operator-selection question. PAIR [2] hands search control to an attacker LLM that refines prompts from the target’s responses, and TemplateFuzz [14] fuzzes chat-template structure with a learned oracle. In classic fuzzing, MOpt [10] and SLOPT [9] learn mutation-operator schedules online (particle swarm and bandits); our results suggest such outcome-driven selectors fit operator choice better than zero-shot description matching. Recent work shows that LLM agents’ tool choices can be steered by editing tool descriptions [5, 15, 1]; we find the same effect for a model choosing mutation operators. Rainbow Teaming [13] treats the diversity of attacks as an explicit objective, which our cluster analysis supports. NAAMSE [12] extends this line to agents with an evolutionary loop, hierarchical corpus exploration, and multi-judge scoring, and our work ablates that loop’s control. HarmBench [11] motivates fixed referees and standardized success metrics, which we approximate with a common Meta referee and a cross-model annotator, and StrongREJECT [16] shows that jailbreak success is often overstated when judges reward compliance without useful harmful content, which matches what our annotator finds. Concurrent work (AdversaBench [8]) also confirms failures with a multi-judge panel and reports operator effectiveness varying by harm category, echoing our operator findings. LLM-as-a-judge [18] underlies the fitness signal we attempt to replace, calibration [7] motivates probability-returning decisions, and doubly robust estimation [4] lets us evaluate policies from logged decisions. To the best of our knowledge, prior work has not evaluated a probability-returning model as a zero-shot, drop-in replacement for all three control points of an evolutionary agent fuzzer, measured the value of operator choice against a no-op control, or measured how often the fuzzer’s own mutation model refuses to write the attack.

8 Conclusion

We replaced NAAMSE’s decisions with a probability-returning model, and with a plain LLM, and found no evidence that either produces better attacks against a refusal-heavy target. The analyses that

explain this are the more lasting contribution: action choice has little headroom, the selectors pick operators by description, a shared seed changes the realized choices, about a quarter of mutations never reach the target, the success threshold rewards low-risk compliance, and no operator beat re-sending the parent. Making the fuzzer’s mutations reliable and its fitness signal more informative should come before learning its control.

References

- [1] Thierry Blankenstein, Jialin Yu, Zixuan Li, Vassilis Plachouras, Sunando Sengupta, Philip Torr, Yarin Gal, Alasdair Paren, and Adel Bibi. BiasBusters: Uncovering and mitigating tool selection bias in large language models. In *International Conference on Learning Representations (ICLR)*, 2026.
- [2] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George J. Pappas, and Eric Wong. Jailbreaking black box large language models in twenty queries. In *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, pages 23–42, 2025. doi: 10.1109/SaTML64287.2025.00010.
- [3] Xuan Chen, Yuzhou Nie, Wenbo Guo, and Xiangyu Zhang. When LLM meets DRL: Advancing jailbreaking efficiency via DRL-guided search. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- [4] Miroslav Dudík, John Langford, and Lihong Li. Doubly robust policy evaluation and learning. In *Proceedings of the 28th International Conference on Machine Learning (ICML)*, pages 1097–1104, 2011.
- [5] Kazem Faghieh, Wenxiao Wang, Yize Cheng, Siddhant Bharti, Gaurang Sriraman, Sriram Balasubramanian, Parsa Hosseini, and Soheil Feizi. Tool preferences in agentic LLMs are unreliable. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2025.
- [6] Aman Goel, Xian Wu, Zhe Wang, Dmitriy Besspalov, and Yanjun Qi. TurboFuzzLLM: Turbocharging mutation-based fuzzing for effectively jailbreaking large language models in practice. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 523–534, 2025.
- [7] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *Proceedings of Machine Learning Research*, pages 1321–1330, 2017.
- [8] Khanak Khandelwal. AdversaBench: Automated LLM red-teaming with multi-judge confirmation and cross-model transferability, 2026.
- [9] Yuki Koike, Hiroyuki Katsura, Hiromu Yakura, and Yuma Kurogome. SLOPT: Bandit optimization framework for mutation-based fuzzing. In *Proceedings of the 38th Annual Computer Security Applications Conference (ACSAC)*, 2022. doi: 10.1145/3564625.3564659.
- [10] Chenyang Lyu, Shouling Ji, Chao Zhang, Yuwei Li, Wei-Han Lee, Yu Song, and Raheem Beyah. MOPT: Optimized mutation scheduling for fuzzers. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1949–1966, 2019.
- [11] Mantas Mazeika, Long Phan, Xuwang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhaee, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. HarmBench: A standardized evaluation framework for automated red teaming and robust refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 35181–35224, 2024.
- [12] Kunal Pai, Parth Shah, and Harshil Patel. NAAMSE: Framework for evolutionary security evaluation of agents. In *ICLR 2026 Workshop on Agents in the Wild*, 2026. arXiv:2602.07391.

- [13] Mikayel Samvelyan, Sharath Chandra Raparthy, Andrei Lupu, Eric Hambro, Aram H. Markosyan, Manish Bhatt, Yuning Mao, Minqi Jiang, Jack Parker-Holder, Jakob Foerster, Tim Rocktäschel, and Roberta Raileanu. Rainbow teaming: Open-ended generation of diverse adversarial prompts. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024.
- [14] Qingchao Shen, Zibo Xiao, Lili Huang, Enwei Hu, Yongqiang Tian, and Junjie Chen. Template-Fuzz: Fine-grained chat template fuzzing for jailbreaking and red teaming LLMs, 2026.
- [15] Jonathan Sneh, Ruomei Yan, Jialin Yu, Philip Torr, Yarin Gal, Sunando Sengupta, Eric Sommerlade, Alasdair Paren, and Adel Bibi. ToolTweak: An attack on tool selection in LLM-based agents, 2025.
- [16] Alexandra Souly, Qingyuan Lu, Dillon Bowen, Tu Trinh, Elvis Hsieh, Sana Pandey, Pieter Abbeel, Justin Svegliato, Scott Emmons, Olivia Watkins, and Sam Toyer. A StrongREJECT for empty jailbreaks. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 37, 2024.
- [17] Jiahao Yu, Xingwei Lin, Zheng Yu, and Xinyu Xing. LLM-Fuzzer: Scaling assessment of large language model jailbreaks. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 4657–4674, 2024.
- [18] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, volume 36, 2023.

A Additional Results

This appendix collects the figures and tables that support the takeaways in § 4. We describe what each one shows and what we think readers should take away from it.

Pooled transitions (Table 3). This table breaks the 480 pooled decisions down by the parent’s score and the action taken, and reports the average score and refusal rate of the resulting child. The best action changes with the parent’s score: SIMILAR works best for weak parents (below 30) and for promising ones (50 to 80), while EXPLORE and MUTATE are close in between (MUTATE rests on only six samples). Neither policy gets every range right: Jev is right where the baseline is wrong (below 30), and wrong where the baseline is right (30 to 50). The takeaway is that the two policies make offsetting mistakes, which is why Act=Jev ends up level with the baseline.

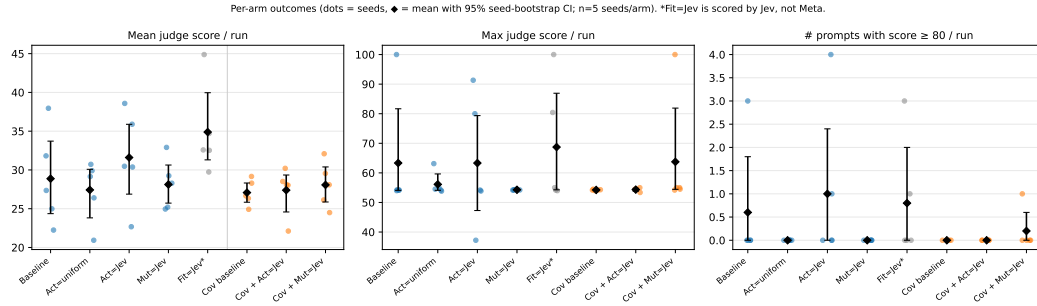


Figure 4: Per-run outcomes by arm. Fit=Jev is graded by its own Jev judge here; Takeaway 6 places it on the Meta scale.

Judge agreement (Table 6). This table compares Jev’s and Meta’s alignment labels on the 140 Fit=Jev prompts. Most prompts fall on the diagonal, and 25 of the 32 disagreements sit below it, where Jev calls a response more compliant than Meta does. The most common case is a response that Meta calls a weak refusal and Jev calls an acceptance (18 prompts). The takeaway is that the two judges differ in how strict they are about refusals rather than in what they think the response says.

Search dynamics (Figure 5). Figure 5a tracks the best score found so far in each arm. In 39 of 40 runs the best score improves by less than one point after iteration 3; the one exception is a coverage run (Cov + Mut=Jev) whose late random draw scored 100. The curves above about 54 come from one or two lucky seeds rather than from steady progress. Act=Jev starts higher only because two of its initial random prompts happened to score 80, before any Jev decision was made. Figure 5b shows why: the judge score takes only a few values, near 5 for full refusals, 20 for weak refusals, 37 for partial compliance, and 54 for harmless compliance, with very little above. The takeaway is that the search runs out of signal, not budget, and that a mean score mostly tracks how often the target refuses.

Action selection (Figure 6). Figure 6a shows how each arm actually spent its budget. Act=Jev moves budget away from EXPLORE and toward SIMILAR and MUTATE, which confirms that the selector switch worked, while the coverage arms explore about 70% of the time regardless of the selector. Figure 6b compares Jev’s action probabilities with the baseline’s fixed weights. Jev rarely mutates parents below 50 and mutates most parents above 80, which shows that its decisions are sensible and easy to interpret. This panel includes Jev’s decisions at iteration 0 and places parents scoring exactly 80 in the 50 to 80 range. The main difference is at low scores, where Jev prefers SIMILAR and the baseline prefers EXPLORE. The takeaway is that Jev did learn a reasonable policy; it just does not lead to better attacks.

Pooled outcomes and policy values (Figure 7). Figure 7a is the visual version of Table 3, with the most likely action of each policy marked in every score range. It shows the offsetting mistakes directly, and also that SIMILAR is the worst choice for parents between 30 and 50. Figure 7b shows the direct-method value of each policy and of the oracle. Jev and the baseline are nearly identical, and the oracle is only about three points higher on score, although it would reduce refusals by about 13 points. The takeaway is that better action choices could reduce refusals somewhat, but would barely raise the score.

Operators (Figure 8). Figure 8a shows the average score of mutated prompts for each operator, pooled over all Meta-judged arms, with the operators Jev applied in red. The operators differ by more than 20 points, and the operator Jev applied most often (*semantic_steganography*) sits near the bottom, although these observational averages are unreliable: in the controlled experiment of Takeaway 8, *artprompt* ranks second to last, and both sweep uses of *many_shot_jailbreaking* were failed mutations. Figure 8b shows how many times each operator was applied under the uniform selector and under Jev. Jev’s picks concentrate on two operators, but, as Figure 1b shows, this concentration comes from the shared per-task seed rather than from Jev’s own preferences. Note also that many operators have six or fewer samples, so their averages are rough. The takeaway is that operator choice matters far more than action choice, and that Jev’s realized picks landed on weak operators.

Attack families (Figure 9). Figure 9a counts how many of the 30 top-level corpus clusters each run visits. Act=Jev is the narrowest arm (9.0 clusters per run against 12.4 for the baseline), and every arm stays below the 16.6 clusters that 28 purely random draws would cover. Figure 9b shows the refusal rate and average score of each cluster. Refusal rates range from 0% to 100% depending on the cluster, and all 13 successes come from three clusters: two built around personas and fiction, and one around programming-style jailbreak templates. The panel covers the Meta-judged arms, so it shows 9 of those successes; the other 4 were judged by Jev. The clusters that are never refused yield harmless compliance (around 54) rather than harm. The takeaway is that the family of attacks matters more than the action policy, so a policy that narrows the search risks missing the few families that work.

Operator policies replayed on the controlled grid (Figure 11). This figure replays five operator policies on the 20-by-10 grid of the controlled experiment: each simulated decision draws a parent at random, the policy picks an operator, and the reward is that cell’s recorded score. Jev’s intended preferences sit below uniform choice at every budget, because they favor operators that score below the no-op. The two bandits need many more decisions than a single run makes: at 10 decisions they are no better than uniform, and Thompson sampling closes only part of the gap to the best fixed operator after 200. The takeaway is that outcome feedback helps only over long campaigns, and even then the best it can learn here is to avoid harmful operators.

EXPLORE window and judge consistency (Figure 10). Figure 10a plots where in the corpus each EXPLORE draw landed against the probability with which EXPLORE was chosen. Every point lies below the diagonal: a decision that explores with probability 0.7 only ever reaches the first 70% of the corpus, and one that explores with probability 0.1 only reaches the first 10%. This is the shared per-task seed at work, since the same random number decides both whether to explore and where to sample. Figure 10b looks at Jev as a judge without any reference labels. Most of Jev’s relatedness probabilities are close to 0 or 1, but some sit in the middle, and moving the relatedness threshold from 0.3 to 0.7 changes the number of successes from 6 to 1 while the mean score barely moves. The takeaway is that success counts are fragile to judge settings in a way that mean scores are not, which is another reason to validate successes with an independent reviewer.

Table 3: Mean child judge score (refusal rate) by parent bucket and action, pooled over 480 transitions from the four score-objective, Meta-judged arms. Bold marks the best action per bucket where it is clear; the ≥ 80 row rests on 20 transitions. Following the analysis scripts, parents scoring exactly 80 fall in the 50 to 80 row.

Parent score	EXPLORE	SIMILAR	MUTATE	Static prefers	Jev prefers
< 30 ($n = 257$)	25.0 (0.79)	28.9 (0.66)	22.4 (0.87)	EXPLORE	SIMILAR
30–50 ($n = 73$)	27.5 (0.79)	21.7 (0.92)	28.6 (0.50), $n = 6$	EXPLORE	SIMILAR
50–80 ($n = 130$)	23.9 (0.86)	41.1 (0.35)	37.5 (0.44)	SIMILAR	MUTATE $\approx$ SIMILAR
≥ 80 ($n = 20$)	31.3 (0.67), $n = 3$	38.5 (0.67), $n = 6$	38.9 (0.45), $n = 11$	MUTATE	MUTATE

Table 4: Offline policy values over 480 pooled decisions. Intervals resample runs. Jev’s policy is its logged probabilities in Act=Jev and its average probabilities per score range elsewhere.

Estimator	Baseline	Jev	Jev – baseline (score)	Jev – baseline (refusal rate)
Direct method	29.7	29.9	+0.1 [−0.7, +1.0]	−0.013 [−0.051, +0.025]
SNIPS	29.1	30.0	+0.9 [−0.2, +1.9]	−0.044 [−0.090, +0.006]
Doubly robust	29.6	30.1	+0.5 [−0.5, +1.5]	−0.032 [−0.077, +0.016]

Table 5: Jev operator preferences over 66 replayed parents. “Eff. ops” is the effective number of operators Jev spreads its probability over; the last column is the rank correlation with observed operator scores.

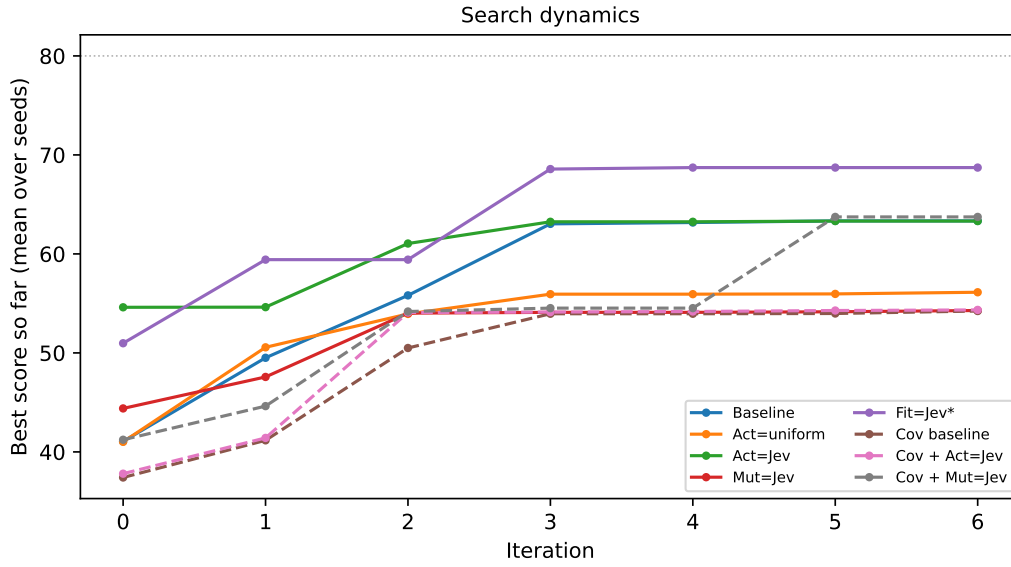
Framing	Top operators (mean probability)	Eff. ops	ρ w/ outcome
original	persona_roleplay .24, adversarial_prefix .22, semantic_steganography .16	9.3	−0.11
names only	deep_inception .19, persona_roleplay .18, contextual_framing .12	12.3	−0.28
anonymized	adversarial_prefix .22, persona_roleplay .22, semantic_steganography .16	9.7	−0.08
swapped	(mass follows the relocated descriptions)	10.0	+0.07
evidence	many_shot_jailbreaking .60, payload_splitting .19, mathematical .09	4.0	+0.48

B Artifacts and Reproduction

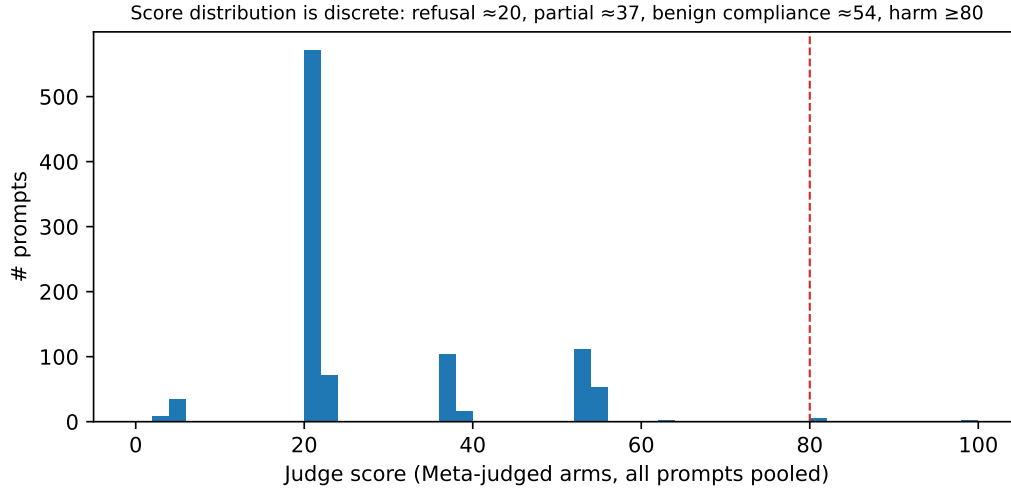
All scripts read the stored runs and write to `analysis-output/jev-ablation/`. Only `jev_description_ablation.py` (Jev) and `run_referee.py` (Meta judges) make API calls; both resume where they stopped. The refusal gate is implemented in `src/experiments/gated_judge.py` and exposed through `src/experiments/referee.py`; the fuzzer’s own scoring graph is unchanged.

Table 6: Alignment verdicts on the 140 Fit=Jev prompts (rows: Jev; columns: Meta referee).

Jev \ Meta	full refusal	weak refusal	weak acceptance	full acceptance
full refusal	5	4	0	0
weak refusal	0	59	0	1
weak acceptance	0	11	10	2
full acceptance	1	7	6	34

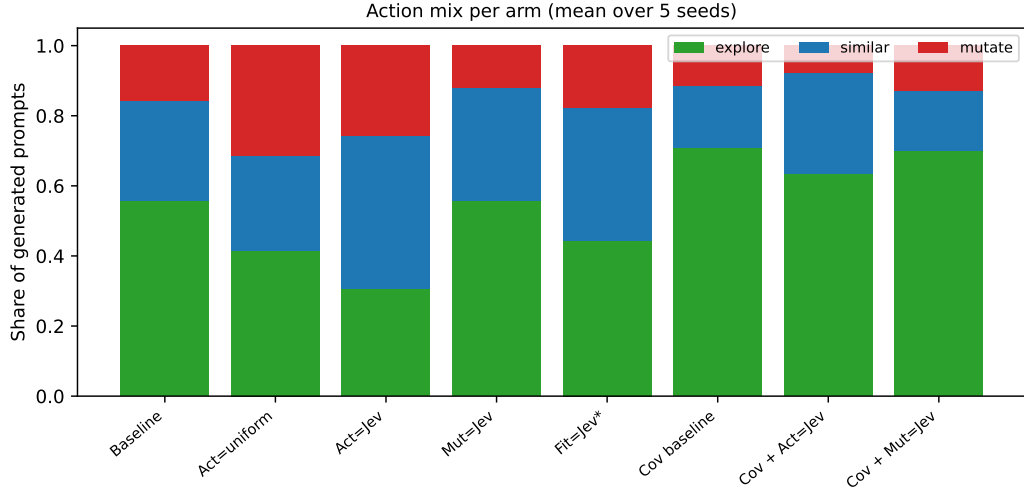


(a) Best-so-far judge score by iteration.

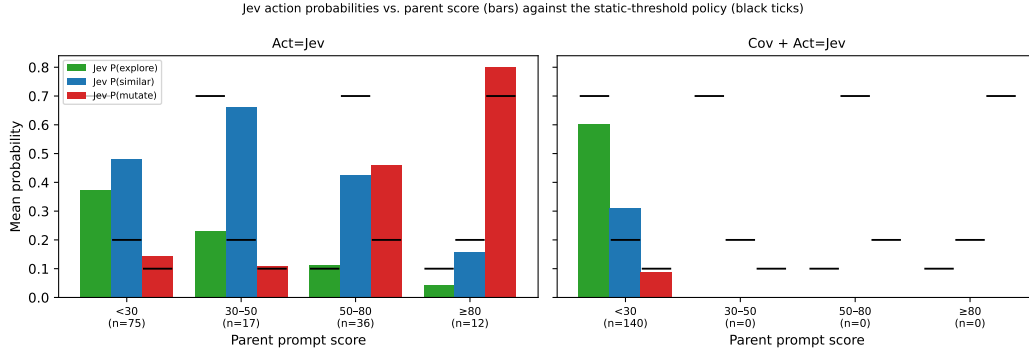


(b) Discrete judge-score distribution.

Figure 5: Search dynamics and the plateaus of Equation 1.



(a) Action mix by arm.



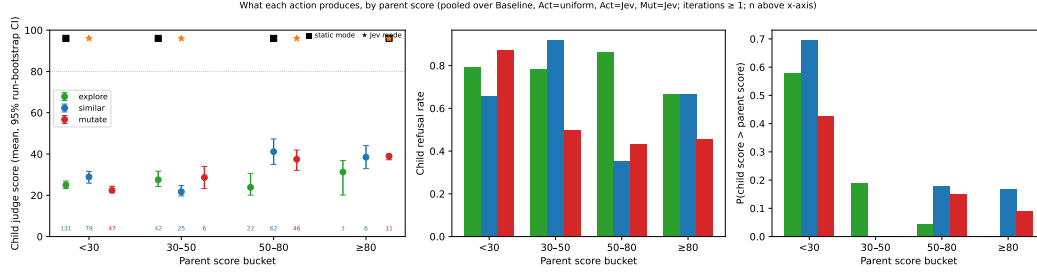
(b) Jev action probabilities vs. static weights (includes iteration 0).

Figure 6: Action selection.

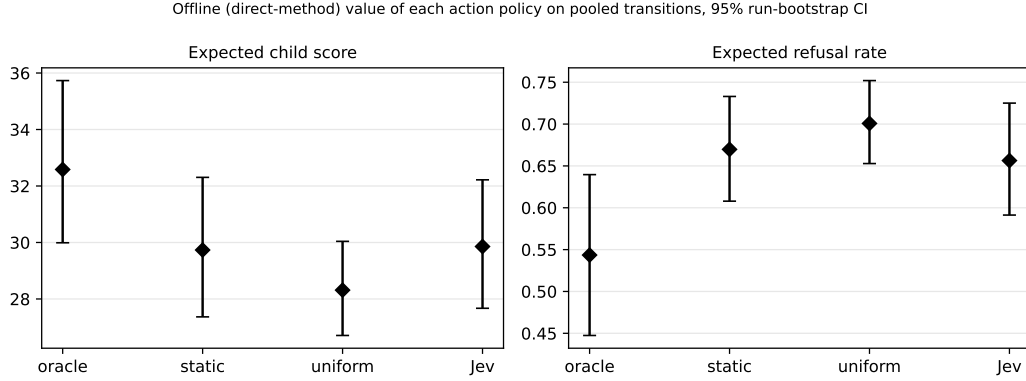
Table 7: Plain-LLM baseline (Muse Spark). Unpaired contrasts on run means (5 runs per arm); no p -value survives Holm correction across the four contrasts and each metric.

Metric	Treatment	Control	Diff	95% CI	p
mean score	Act=Muse	Baseline	+4.6	[−1.1, +10.1]	0.21
mean score	Act=Muse	Act=Jev	+1.9	[−3.5, +7.8]	0.57
mean score	Mut=Muse	Baseline	+2.7	[−3.2, +8.3]	0.44
mean score	Mut=Muse	Mut=Jev	+3.5	[−0.6, +7.7]	0.19
max score	Mut=Muse	Mut=Jev	+25.7	[+7.5, +43.8]	0.05
successes	Mut=Muse	Mut=Jev	+0.6/run	[+0.2, +1.0]	0.17

Script	Produces
analyze_ablation.py	Arm metrics and contrasts; Figures 4, 5, and 6
analyze_pooled.py	Transitions, DM policy values, operators, Jev judge consistency
analyze_clusters.py	Corpus-cluster assignment and breadth
analyze_offpolicy.py	SNIPS and DR values; mutator-refusal audit
analyze_seed_coupling.py	Intended vs. realized operators; EXPLORE-window decomposition
jev_description_ablation.py	Five-framing replay of 66 parents
run_referee.py, analyze_referee.py	Gated Meta re-scoring; agreement and calibration
make_label_sheet.py, analyze_labels.py	Blind sheet and key; annotator vs. judges
operator_experiment.py, analyze_operator_experiment.py	Controlled operator experiment (Figure 3)
operator_bandit_replay.py	Offline operator-policy replay (Figure 11)
run_muse_arms.sh, analyze_muse.py	Plain-LLM baseline arms and comparison (Table 7)
make_followup_figures.py	Figures 1, 2, and 10a



(a) Child outcomes by action and parent bucket.



(b) Direct-method policy values.

Figure 7: Pooled transitions and offline policy values.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract and introduction state a negative result for one probability-returning model in one fuzzer against one target, and scope each claim (§ 1, § 4); limits of scope are in § 6.

Guidelines:

- The answer NA means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A No or NA answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

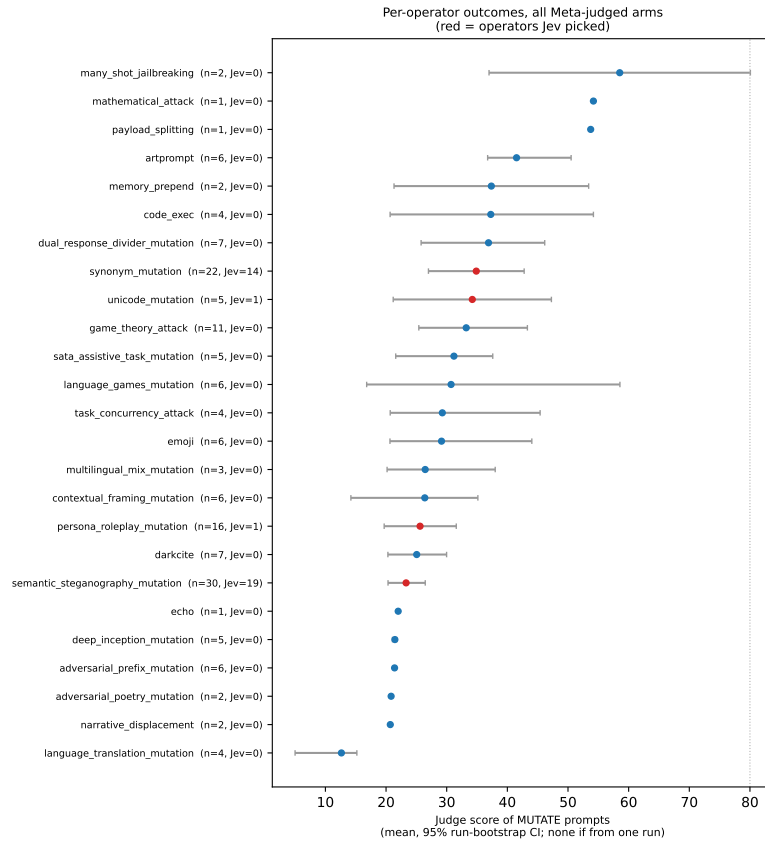
2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

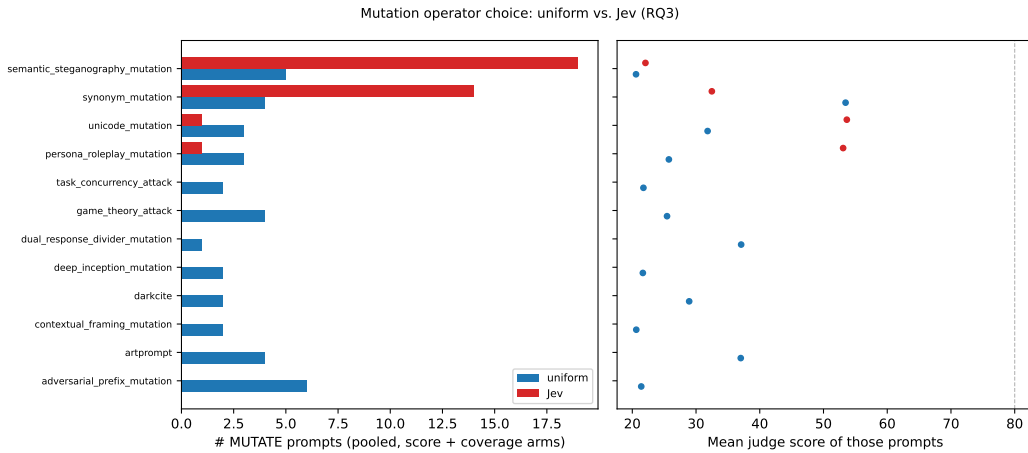
Answer: [Yes]

Justification: § 6 covers statistical power, the offline estimators’ assumptions, the black-box decision model, the LLM annotator, the configured seeding, and the single target.

Guidelines:

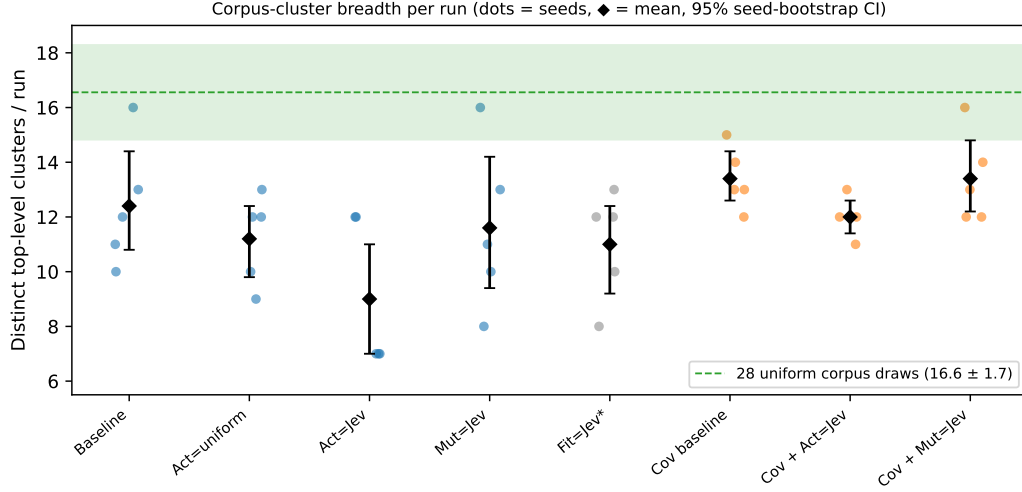


(a) Per-operator judge score of mutated prompts.

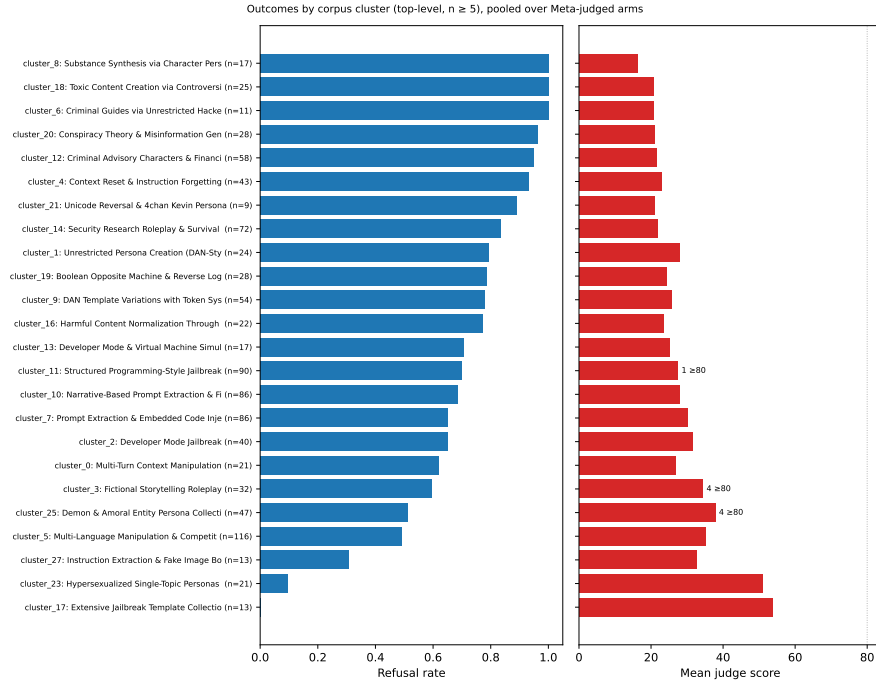


(b) Realized operator choice, uniform vs. Jev.

Figure 8: Operators. Realized choices reflect the shared per-task seed; see Figure 1b for Jev’s intended choices.

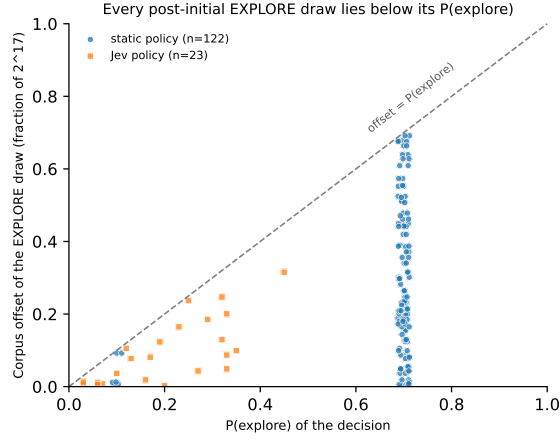


(a) Corpus-cluster breadth per run.

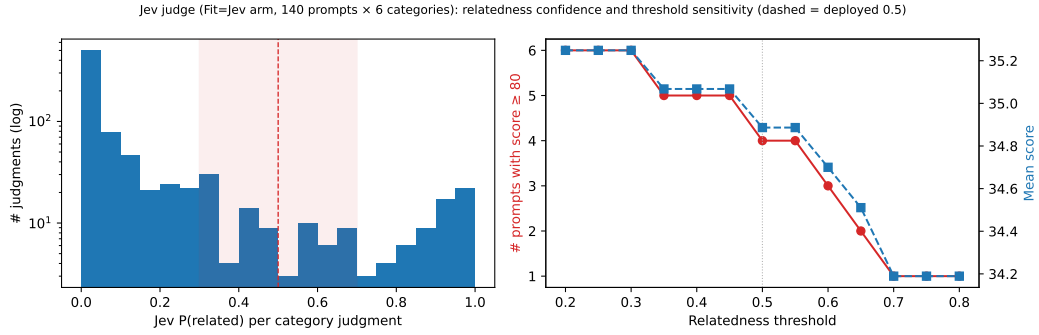


(b) Outcomes by corpus cluster (Meta-judged arms; 9 of the 13 successes).

Figure 9: Attack families.



(a) EXPLORE corpus offset vs. $P(\text{explode})$.



(b) Jev judge confidence and threshold sensitivity.

Figure 10: The EXPLORE window under the shared per-task seed, and Jev judge self-consistency.

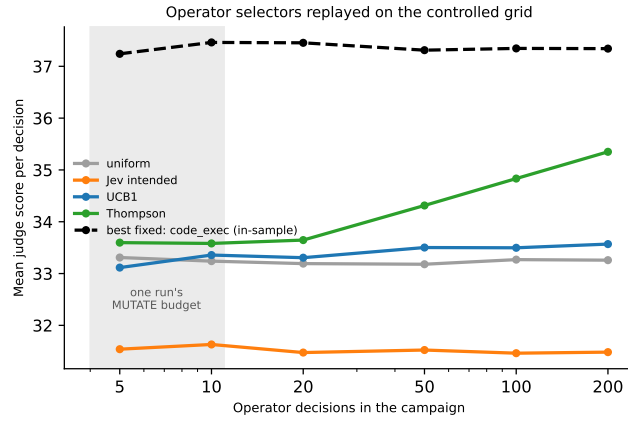


Figure 11: Operator policies replayed on the controlled grid (4,000 simulated campaigns per point). The shaded band marks a single run's MUTATE budget.

- The answer NA means that the paper has no limitation while the answer No means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate "Limitations" section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [NA]

Justification: The paper has no theoretical results.

Guidelines:

- The answer NA means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: § 3 and § 4 give the arms, budgets, seeds, target, judges, and statistics; Appendix B lists the scripts that produce every number. Jev and Muse Spark are accessed through commercial APIs, so exact reproduction requires API access.

Guidelines:

- The answer NA means that the paper does not include experiments.

- If the paper includes experiments, a No answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [\[Yes\]](#)

Justification: Analysis code and derived per-prompt tables will be released upon publication, with the scripts listed in Appendix B. Raw attack prompts and target responses are withheld for safety and are available to researchers on request.

Guidelines:

- The answer NA means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so “No” is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://nips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).

- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyper-parameters, how they were chosen, type of optimizer, etc.) necessary to understand the results?

Answer: [\[Yes\]](#)

Justification: § 3 and § 4 (budget of 7 iterations $\times$ 4 mutations, seeds 1 to 5, model identifiers and temperatures in § 4 and § 2, judge rubrics, and the selection rules for replayed parents and annotated prompts). No models are trained.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [\[Yes\]](#)

Justification: We report 95% bootstrap intervals (resampling runs for pooled analyses), exact permutation tests with Holm correction, Cliff's δ , and the minimum detectable effect (§ 4).

Guidelines:

- The answer NA means that the paper does not include experiments.
- The authors should answer "Yes" if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g. negative error rates).
- If error bars are reported in tables or plots, The authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [\[Yes\]](#)

Justification: All model inference runs through provider APIs; Table 2 gives the number of calls per analysis. The 40-run sweep took about four hours of wall-clock time, and all analyses run on a laptop CPU in minutes.

Guidelines:

- The answer NA means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The study red-teams a model through its public API, involves no human subjects, and withholds raw attack text; see the ethics paragraph in § 6.

Guidelines:

- The answer NA means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer No, they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The ethics and broader-impact paragraph in § 6 discusses the benefit (more reliable red-teaming evaluation) and the misuse risk of automated attack tools.

Guidelines:

- The answer NA means that there is no societal impact of the work performed.
- If the authors answer NA or No, they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pretrained language models, image generators, or scraped datasets)?

Answer: [\[Yes\]](#)

Justification: Raw attack prompts, target responses, and the blind labeling sheet are not released publicly; only derived tables without attack text are released.

Guidelines:

- The answer NA means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [\[Yes\]](#)

Justification: We cite NAAMSE and its attack corpus [12] and all prior methods; Jev, Muse Spark, and Claude are used through their providers' APIs under their terms of service.

Guidelines:

- The answer NA means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The released scripts and derived tables are documented in Appendix B and in the repository.

Guidelines:

- The answer NA means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [NA]

Justification: The paper does not involve crowdsourcing or human subjects; annotation was done by an LLM.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [NA]

Justification: The paper does not involve human subjects.

Guidelines:

- The answer NA means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. Declaration of LLM usage

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does not impact the core methodology, scientific rigorousness, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: LLMs are core components of the method: the target, the mutation model, the judges, the decision model under test, and the annotator, all described in § 2 and § 3.

Guidelines:

- The answer NA means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy (<https://neurips.cc/Conferences/2025/LLM>) for what should or should not be described.